

CETAM

CURSO TÉCNICO DE NÍVEL MÉDIO
EM INFORMÁTICA

MATERIAL DIDÁTICO

FUNÇÕES EM PYTHON

Componente Curricular Linguagem de Programação (Python)
Carga horária 60 horas
Conteúdo Funções • parâmetros • argumentos • return • modularização e reutilização de código

Prof. Esp. Kelrison Coêlho

2026

1. O QUE É UMA FUNÇÃO?

Uma função é um bloco de código criado para realizar uma tarefa específica. Ela permite organizar o programa e evitar a repetição do mesmo código.

```
def mostrar_menu():  
    print("1 - Cadastrar livro")  
    print("2 - Listar livros")  
    print("0 - Sair")
```

```
mostrar_menu()
```

Observe

A função é definida uma vez e pode ser chamada sempre que essa tarefa for necessária.

Estrutura básica

```
def nome_da_funcao(parametro):  
    # instruções  
    return resultado
```

A indentação define quais instruções pertencem à função.

Como a função funciona?

Uma função passa pela chamada, execução do processamento e obtenção do resultado.

2. PARÂMETROS E ARGUMENTOS

Parâmetros são os dados que uma função espera receber. Argumentos são os valores enviados quando a função é chamada.

Termo	Exemplo	O que significa?
Parâmetro	nome	É definido na criação da função.
Argumento	"Python"	É enviado na chamada da função.

Exemplo: dados de um livro

```
def apresentar_livro(titulo, autor):  
    print("Título:", titulo)  
    print("Autor:", autor)
```

```
apresentar_livro("Dom Casmurro", "Machado de Assis")
```

A função pode receber mais de um parâmetro. A ordem dos argumentos corresponde à ordem dos parâmetros.

Outro exemplo

```
def calcular_total(preco, quantidade):  
    return preco * quantidade
```

```
valor = calcular_total(25, 3)  
print(valor)
```

Boa prática

Use nomes claros como `preco` e `quantidade`, evitando nomes genéricos quando eles não ajudam a entender o código.

3. O COMANDO return

O return envia um resultado de volta para o ponto onde a função foi chamada. O resultado pode ser guardado em uma variável ou usado em outra operação.

```
def somar(a, b):  
    resultado = a + b  
    return resultado
```

```
valor = somar(10, 5)  
print(valor)
```

print()	return
Mostra uma informação na tela.	Devolve um valor para quem chamou a função.
Útil para exibição.	Útil para continuar o processamento.

Função com decisão

```
def verificar_estoque(quantidade):  
    if quantidade > 0:  
        return "Disponível"  
    return "Indisponível"
```

```
situacao = verificar_estoque(5)  
print(situacao)
```

Uma função também pode executar decisões e devolver resultados diferentes.

Atenção

Uma função que apenas executa uma ação pode não precisar de return. Quando o resultado será usado em outra parte do programa, o return é importante.

4. FUNÇÕES APLICADAS AO CADASTRO DE LIVROS

Vamos separar cada responsabilidade do sistema de biblioteca em uma função.

```
biblioteca = []
```

```
def cadastrar_livro():  
    titulo = input("Título: ")  
    autor = input("Autor: ")  
    livro = [titulo, autor]  
    biblioteca.append(livro)
```

```
def listar_livros():  
    for contador in biblioteca:  
        print("Título:", contador[0])  
        print("Autor:", contador[1])
```

```
cadastrar_livro()  
listar_livros()
```

A lista biblioteca continua armazenando os dados, enquanto as funções controlam as operações realizadas sobre ela.

Função	Responsabilidade
cadastrar_livro()	Receber os dados e adicionar o livro à lista.
listar_livros()	Percorrer a lista e mostrar os livros.
buscar_livro()	Localizar um livro conforme um critério.
remover_livro()	Retirar um livro da lista.

Próximo passo

O menu principal pode apenas chamar as funções. Isso reduz a quantidade de código concentrada em um único lugar e facilita a manutenção.

5. ORGANIZANDO UM PROGRAMA COM MENU

O menu recebe a opção do usuário e chama a função correspondente.

```
biblioteca = []

def cadastrar_livro():
    titulo = input("Título: ")
    autor = input("Autor: ")
    biblioteca.append([titulo, autor])

def listar_livros():
    for contador in biblioteca:
        print(contador[0], "-", contador[1])

while True:
    print("\n1 - Cadastrar")
    print("2 - Listar")
    print("0 - Sair")
    opcao = input("Escolha: ")

    if opcao == "1":
        cadastrar_livro()
    elif opcao == "2":
        listar_livros()
    elif opcao == "0":
        break
    else:
        print("Opção inválida")
```

Observe a divisão de responsabilidades: o while controla a repetição do menu, enquanto cada função executa uma tarefa específica.

Vantagens dessa organização

- facilita a leitura do código;
- reduz a repetição de instruções;
- facilita localizar erros;
- permite alterar uma funcionalidade sem mexer em todo o programa.

Próximo passo

Depois que o programa funcionar, procure partes repetidas ou muito extensas e avalie se podem ser transformadas em funções.

6. PRÁTICA — CRIE SUAS PRÓPRIAS FUNÇÕES

Aplique o conceito no sistema de biblioteca desenvolvido em sala.

Desafio 1: crie `calcular_total()` que receba preço e quantidade e retorne o total.

Desafio 2: crie `cadastrar_livro()` para adicionar um livro à lista.

Desafio 3: crie `listar_livros()` para exibir todos os registros.

Desafio 4: crie `buscar_livro()` para procurar um título na lista.

Desafio 5: crie `remover_livro()` para retirar um registro da lista.

Teste cada função separadamente antes de integrá-la ao menu.

```
def calcular_total(preco, quantidade):  
    return preco * quantidade
```

```
resultado = calcular_total(20, 4)  
print(resultado)
```

Regra de ouro

Se uma parte do programa possui uma responsabilidade específica e pode ser reutilizada, pense: “Isso pode virar uma função?”

Próximo passo: refatorar o projeto, identificando partes que podem ser organizadas em funções.